

Database Management Systems

PROJECT REPORT

Design and Implementation of a Database Management System for Hostel Accommodation and Student Services Management System

Submitted by

Team Member	Roll Number
Mohammed Amaan Khan	25WU0101075
Arslan Alam	25WU0101016
Jaswant Reddy	25WU0101056

Academic Year 2026-2027

Abstract

Most college hostels still run on registers and spreadsheets. Beds get allotted twice, complaints about fans or Wi-Fi are written on slips that go missing, and nobody can say quickly who has paid this month's rent. This project designs and implements a relational database that keeps hostel rooms, room allotment, student service requests and rent payments in one place.

The database has ten tables in Third Normal Form, protected by primary keys, foreign keys, unique and check constraints. Two stored procedures handle room allotment and service requests, four triggers stop invalid allotments and payments and move every service request through a fixed order of steps, and three views report occupancy, open requests and rent paid. The database is implemented in MySQL 8, and a Node.js web interface connected to MySQL demonstrates the complete workflow for students, wardens and maintenance staff.

Keywords: DBMS, MySQL, hostel management, room allocation, service requests, triggers, stored procedures, normalization.

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Requirements
5. System Design
6. Normalization
7. Database Implementation
8. Workflow of the System
9. User Interface
10. Testing and Results
11. Conclusion and Future Scope

References

Appendix A: Report Queries

Appendix B: Source Code

1. Introduction

A hostel office answers the same questions every day. Which room still has a free bed? Who is staying in A101? Has the plumber looked at the leaking tap that was reported last week? Which residents have not paid this month's rent? When the answers live in separate registers, each one takes time to find and is often out of date.

A Database Management System (DBMS) puts all of this in one place. Once rooms, residents, requests and payments are stored together, rules such as "a double room cannot hold three students" or "a girl cannot be allotted a room in a boys' hostel" can be enforced by the database itself, instead of depending on someone remembering to check. This project builds such a system for the hostels of a university campus.

2. Problem Statement

Design and implement a database for a Hostel Accommodation and Student Services Management System that:

- stores hostels, rooms, students, departments and staff without redundancy;
- allots rooms to students and keeps the history of every stay;
- accepts service requests (electrical, plumbing, housekeeping, internet, mess, laundry) and tracks each one until it is resolved;
- records monthly rent payments and shows who still has to pay; and
- reports occupancy, open requests and fee collection.

The problems with the manual process are:

- Beds are allotted twice, or a room is given to a student who already has one.
- Complaints are lost, and there is no record of who worked on them or when they were closed.
- Rent receipts are kept on paper, so the list of defaulters has to be prepared by hand every month.
- Free beds cannot be found without walking around or calling every warden.

3. Objectives

1. To design an Entity-Relationship model and a normalized relational schema for the system.
2. To implement the schema in MySQL with proper integrity constraints.
3. To enforce room allotment rules (capacity, hostel gender, one active room per student) inside the database.
4. To implement a service request workflow (Open, Assigned, In Progress, Resolved) with a full history of every step.
5. To use triggers and stored procedures so that business rules hold however the data is entered.

6. To provide views and queries that report occupancy, open requests and rent dues.
7. To build a user interface connected to MySQL that demonstrates the complete workflow.

4. Scope and Requirements

4.1 Users

User	Responsibilities
Student (resident)	View own room and roommates, raise and cancel service requests, follow each request, pay monthly rent.
Warden (hostel office)	Allot and vacate rooms, check occupancy and free beds, see rent dues for any month, watch all service requests.
Service staff	Electrician, plumber, housekeeping, network technician and mess supervisor. Each sees the requests of their trade and moves them to the next step with remarks.

4.2 Functional requirements

- Maintain master data: departments, staff, hostels, rooms, students and service types.
- Allot a room to a student; refuse a full room, a hostel of the other gender, or a student who already has a room.
- Vacate a room and keep the old stay as history.
- Raise a service request; only a current resident can raise one, and only one open request of each type is allowed.
- Move a request through Assigned, In Progress and Resolved, one step at a time, and store who did each step and when.
- Record rent payments; the amount must equal the rent of the student's current room, and a month can be paid only once.
- Report occupancy, free beds, open requests, request history, rent dues and collection.

4.3 Non-functional requirements

- **Integrity:** invalid data (for example a check-out date before check-in, or a room with capacity 5) must be rejected by the database.
- **Auditability:** every change in a service request is stored with the staff member, remarks and time.
- **Consistency:** no two payments for the same student and month; no two rooms with the same number in one hostel.
- **Performance:** indexes on the columns used most in queries.

4.4 Software used

Component	Tool
DBMS	MySQL 8.0
Database client	MySQL Workbench / mysql command-line client
Back end	Node.js with Express and the mysql2 driver
Front end	HTML, CSS and JavaScript web page
Diagrams	Entity-Relationship diagram of the final schema

5. System Design

5.1 Entities and relationships

The system has ten entities. The many-to-many relationship between Student and Room is resolved by Allocation, which also keeps the dates of each stay. Request_Update links Service_Request and Staff and stores each step of the service workflow.

Relationship	Cardinality / Meaning
Department and Student	1 : N. A department has many students.
Staff (warden) and Hostel	1 : 1. Each hostel is managed by one warden.
Hostel and Room	1 : N. A hostel has many rooms.
Student and Room	M : N through Allocation. A student can stay in different rooms over the years; a room holds several students.
Student and Service_Request	1 : N. A student can raise many requests.
Service_Type and Service_Request	1 : N. Each request belongs to one type.
Service_Request and Request_Update	1 : N (at most 3). One row per step: Assigned, In Progress, Resolved.
Staff and Request_Update	1 : N. A staff member records many steps.
Student and Fee_Payment	1 : N. One payment per student per month.

5.2 ER diagram

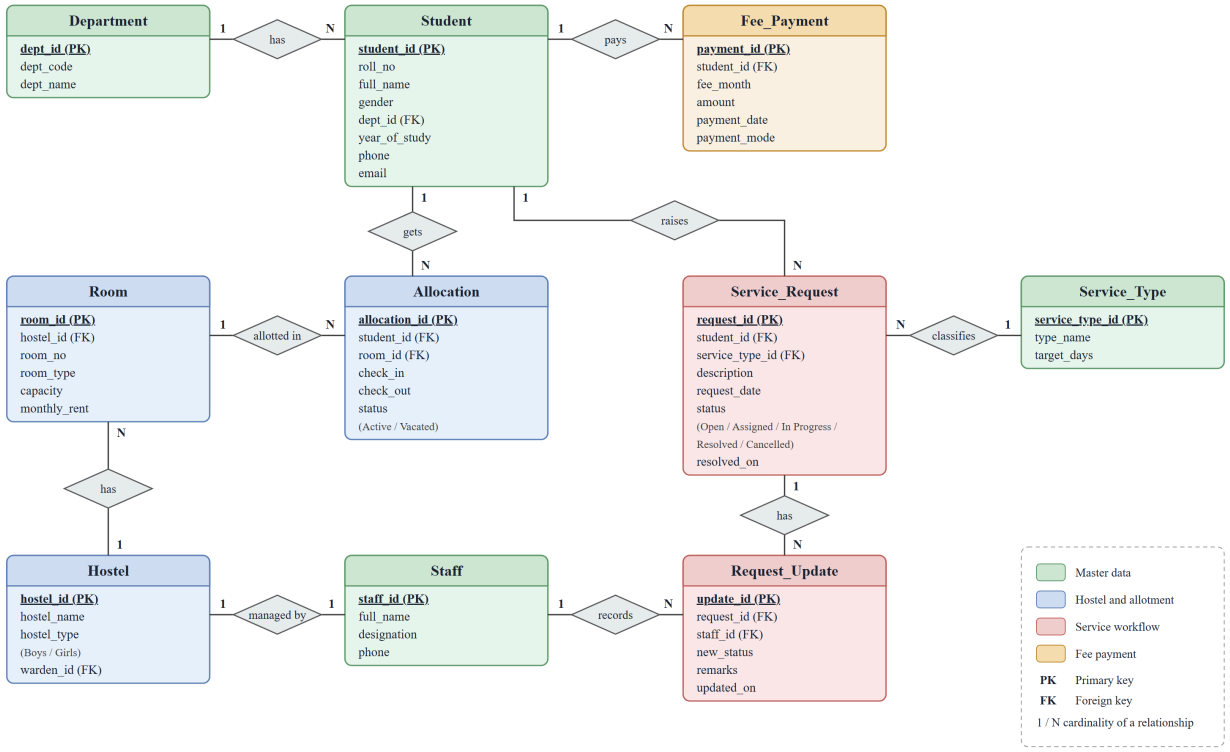


Figure 5.1: ER diagram of the system. Master data is shown in green, hostel and allotment in blue, the service workflow in red and fee payment in yellow.

5.3 Relational schema

Primary keys are underlined. The relational schema, with foreign keys marked, is:

DEPARTMENT(dept_id, dept_code, dept_name)

STAFF(staff_id, full_name, designation, phone)

HOSTEL(hostel_id, hostel_name, hostel_type, warden_id*)

ROOM(room_id, hostel_id*, room_no, room_type, capacity, monthly_rent)

STUDENT(student_id, roll_no, full_name, gender, dept_id*, year_of_study, phone, email)

ALLOCATION(allocation_id, student_id*, room_id*, check_in, check_out, status)

SERVICE_TYPE(service_type_id, type_name, target_days)

SERVICE_REQUEST(request_id, student_id*, service_type_id*, description, request_date, status, resolved_on)

REQUEST_UPDATE(update_id, request_id*, staff_id*, new_status, remarks, updated_on)

FEE_PAYMENT(payment_id, student_id*, fee_month, amount, payment_date, payment_mode)

* marks a foreign key. The full schema with every foreign key target is in Presentation/Relational_Schema.pdf.

5.4 Data dictionary of the main tables

Table	Column	Description and constraint
room	(hostel_id, room_no)	UNIQUE; a room number is used once per hostel

Table	Column	Description and constraint
room	capacity	CHECK between 1 and 3
room	monthly_rent	CHECK monthly_rent > 0
hostel	hostel_type	ENUM: Boys, Girls
hostel	warden_id	FK to staff; UNIQUE, so a warden manages one hostel
student	roll_no, email	UNIQUE; NOT NULL
student	year_of_study	CHECK between 1 and 4
allocation	status	ENUM: Active, Vacated
allocation	check_out	CHECK check_out IS NULL OR check_out >= check_in
service_request	status	ENUM: Open, Assigned, In Progress, Resolved, Cancelled
service_type	target_days	Days within which a request of this type should be resolved
request_update	(request_id, new_status)	UNIQUE; each step happens once per request
fee_payment	(student_id, fee_month)	UNIQUE; one payment per student per month
fee_payment	fee_month	CHECK format YYYY-MM
staff	phone	UNIQUE; CHECK 10 digits starting with 6 to 9

6. Normalization

First Normal Form (1NF). All attributes hold single atomic values, there are no repeating groups, and every table has a primary key. A student's payments are not stored as a list in one column; each payment is a separate row in Fee_Payment, and each step of a request is a separate row in Request_Update.

Second Normal Form (2NF). There are no partial dependencies. Every table has a single-column surrogate key, and the composite candidate keys, (hostel_id, room_no) in Room and (student_id, fee_month) in Fee_Payment, determine the whole row. The rent of a room depends on the room itself, not on the hostel alone or the room number alone.

Third Normal Form (3NF). There are no transitive dependencies. Hostel names and wardens are stored once in Hostel and reached from Room through hostel_id. The rent is stored once in Room and is not copied into Allocation. Department names and service type targets are kept in their own tables, so a request stores only service_type_id and not the type name or its target.

As a result there is no update, insertion or deletion anomaly. If the rent of A101 changes, only one row in Room is updated. A new service type can be added before any request of that type exists, and deleting a request does not remove the service type.

7. Database Implementation

The complete script is in the file hostel_management_system.sql. It creates the database, tables, indexes, views, procedures, triggers, sample data and the report queries. The main parts are shown below.

7.1 Table creation (DDL)

The two tables at the centre of the two workflows, room allotment and service requests:

```
CREATE TABLE allocation (
    allocation_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id    INT NOT NULL,
    room_id       INT NOT NULL,
    check_in      DATE NOT NULL,
    check_out     DATE NULL,
    status        ENUM('Active','Vacated') NOT NULL DEFAULT 'Active',
    CONSTRAINT chk_alloc_dates CHECK (check_out IS NULL OR check_out >= check_in),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
    FOREIGN KEY (room_id)    REFERENCES room(room_id)
);
```

```
CREATE TABLE request_update (
    update_id   INT AUTO_INCREMENT PRIMARY KEY,
    request_id  INT NOT NULL,
    staff_id    INT NOT NULL,
    new_status  ENUM('Assigned','In Progress','Resolved') NOT NULL,
    remarks     VARCHAR(200),
    updated_on  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (request_id, new_status),
    FOREIGN KEY (request_id) REFERENCES service_request(request_id) ON DELETE CASCADE,
    FOREIGN KEY (staff_id)   REFERENCES staff(staff_id)
);
```

7.2 Views

The view v_room_occupancy counts the active allocations of every room, so the number of free beds is always computed from the data and never typed in by hand.

```
CREATE VIEW v_room_occupancy AS
SELECT r.room_id, h.hostel_name, h.hostel_type, r.room_no, r.room_type,
       r.capacity,
       COUNT(a.allocation_id)              AS occupied,
       r.capacity - COUNT(a.allocation_id) AS free_beds,
       r.monthly_rent
FROM room r
JOIN hostel h          ON h.hostel_id = r.hostel_id
LEFT JOIN allocation a ON a.room_id = r.room_id AND a.status = 'Active'
GROUP BY r.room_id, h.hostel_name, h.hostel_type, r.room_no, r.room_type,
         r.capacity, r.monthly_rent;
```

Free beds = capacity - number of Active allocations of the room.

Two more views are used by the interface: v_open_requests lists requests that are not closed yet with the student's room, and v_fee_summary shows the months paid and the total rent paid by every current resident.

7.3 Stored procedures

The procedure sp_raise_request refuses a request from a student who is not a current resident and a second open request of the same type. Only a valid request is inserted, with status Open.

```
CREATE PROCEDURE sp_raise_request(
    IN p_student_id INT, IN p_service_type_id INT,
    IN p_description VARCHAR(200), IN p_request_date DATE)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM allocation
                   WHERE student_id = p_student_id AND status = 'Active') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only a current hostel resident can
raise a request';
    END IF;
    IF EXISTS (SELECT 1 FROM service_request
               WHERE student_id = p_student_id
                 AND service_type_id = p_service_type_id
                 AND status IN ('Open','Assigned','In Progress')) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'An open request of this type already
exists';
    END IF;
    INSERT INTO service_request(student_id, service_type_id, description, request_date)
    VALUES (p_student_id, p_service_type_id, p_description, p_request_date);
END
```

The second procedure, sp_allocate_room, checks that the student and room exist and inserts the allocation. The allotment rules themselves sit in a trigger, so that a direct INSERT into allocation is held to the same rules.

7.4 Triggers

The BEFORE INSERT trigger on allocation blocks an allotment into a full room, into a hostel of the other gender, or for a student who already has an active room.

```
CREATE TRIGGER trg_allocation_before_insert BEFORE INSERT ON allocation
FOR EACH ROW
BEGIN
    DECLARE v_capacity TINYINT;
    DECLARE v_occupied INT;
    DECLARE v_hostel_type VARCHAR(5);
    DECLARE v_gender VARCHAR(6);

    IF NEW.status = 'Active' THEN
        IF EXISTS (SELECT 1 FROM allocation
                   WHERE student_id = NEW.student_id AND status = 'Active') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student already has an active
room';
        END IF;

        SELECT r.capacity, h.hostel_type INTO v_capacity, v_hostel_type
        FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
        WHERE r.room_id = NEW.room_id;

        SELECT gender INTO v_gender FROM student WHERE student_id = NEW.student_id;
        IF (v_hostel_type = 'Boys' AND v_gender <> 'Male')
           OR (v_hostel_type = 'Girls' AND v_gender <> 'Female') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Hostel type does not match
student gender';
```

```
        END IF;

        SELECT COUNT(*) INTO v_occupied FROM allocation
        WHERE room_id = NEW.room_id AND status = 'Active';
        IF v_occupied >= v_capacity THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room is full';
        END IF;
    END IF;
END
```

The BEFORE INSERT trigger on request_update makes the steps happen in order, and the AFTER INSERT trigger copies the new status to the request and stamps the resolved date.

```
CREATE TRIGGER trg_update_before_insert BEFORE INSERT ON request_update
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(15);
    SELECT status INTO v_status FROM service_request WHERE request_id = NEW.request_id;

    IF v_status IN ('Resolved','Cancelled') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Service request is already closed';
    END IF;
    IF NEW.new_status = 'Assigned' AND v_status <> 'Open' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request is already assigned';
    END IF;
    IF NEW.new_status = 'In Progress' AND v_status <> 'Assigned' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request must be assigned before work
starts';
    END IF;
    IF NEW.new_status = 'Resolved' AND v_status <> 'In Progress' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request must be In Progress before it
is resolved';
    END IF;
END

CREATE TRIGGER trg_update_after_insert AFTER INSERT ON request_update
FOR EACH ROW
BEGIN
    IF NEW.new_status = 'Resolved' THEN
        UPDATE service_request
        SET status = 'Resolved', resolved_on = DATE(NEW.updated_on)
        WHERE request_id = NEW.request_id;
    ELSE
        UPDATE service_request SET status = NEW.new_status
        WHERE request_id = NEW.request_id;
    END IF;
END
```

A fourth trigger, trg_fee_before_insert, accepts a payment only from a current resident and only for the exact rent of their room.

7.5 Indexes

Indexes were created on allocation(room_id, status), allocation(student_id, status), service_request(status) and fee_payment(fee_month), because the occupancy view, the allotment trigger, the open request list and the monthly dues report filter on these columns.

7.6 Tables with sample data

The script loads the sample data below. Allotments are made through sp_allocate_room and requests through sp_raise_request, and the request_update rows are inserted one by one, so the triggers run on the sample data exactly as they do in the application. The tables are shown in the state after the script has run. The phone and email columns of student are not shown.

Table: department

dept_id	dept_code	dept_name
1	CSE	Computer Science and Engineering
2	AIML	Artificial Intelligence and Machine Learning
3	ECE	Electronics and Communication Engineering
4	MECH	Mechanical Engineering
5	BBA	Business Administration

Table: staff

staff_id	full_name	designation	phone
1	Mr. Ramesh Kumar	Warden	9876500001
2	Mr. Suresh Nair	Warden	9876500002
3	Mrs. Lakshmi Iyer	Warden	9876500003
4	Mrs. Kavitha Pillai	Warden	9876500004
5	Mr. Vijay Singh	Warden	9876500005
6	Mr. Prakash Yadav	Electrician	9876500006
7	Mr. Imran Shaikh	Plumber	9876500007
8	Ms. Rekha Bai	Housekeeping	9876500008
9	Mr. Naveen Reddy	Network Technician	9876500009
10	Mr. Gopal Rao	Mess Supervisor	9876500010

Table: hostel

hostel_id	hostel_name	hostel_type	warden_id
1	Aryabhata Block	Boys	1
2	Bhaskara Block	Boys	2
3	Gargi Block	Girls	3
4	Maitreyi Block	Girls	4
5	Raman Block	Boys	5

Table: room

room_id	hostel_id	room_no	room_type	capacity	monthly_rent
1	1	A101	Double	2	4500.00
2	1	A102	Triple	3	3500.00
3	2	B201	Single	1	6500.00

room_id	hostel_id	room_no	room_type	capacity	monthly_rent
4	3	G101	Double	2	4500.00
5	3	G102	Single	1	6500.00
6	4	M301	Triple	3	3500.00
7	5	R105	Double	2	4800.00
8	5	R106	Single	1	6000.00

Table: student

student_id	roll_no	full_name	gender	dept_id	year_of_study
1	CS2401	Arjun Mehta	Male	1	2
2	CS2402	Rohan Das	Male	1	2
3	AI2403	Priya Sharma	Female	2	2
4	AI2404	Sneha Reddy	Female	2	3
5	EC2405	Karan Patel	Male	3	1
6	ME2406	Divya Menon	Female	4	4
7	BB2407	Farhan Qureshi	Male	5	1
8	CS2408	Ananya Gupta	Female	1	1

Table: allocation

allocation_id	student_id	room_id	check_in	check_out	status
1	6	6	2025-07-10	2026-05-30	Vacated
2	1	1	2026-07-15		Active
3	2	1	2026-07-15		Active
4	3	4	2026-07-16		Active
5	4	5	2026-07-16		Active
6	5	3	2026-07-20		Active
7	7	7	2026-07-22		Active
8	8	4	2026-08-01		Active

Row 1 is Divya Menon's stay of the last academic year, which is closed. Rows 2 to 8 were inserted by sp_allocate_room. A101 and G101 are now full, so a third student cannot be added to either.

Table: service_type

service_type_id	type_name	target_days
1	Electrical	2
2	Plumbing	2
3	Housekeeping	1
4	Internet	1
5	Mess	3
6	Laundry	2

Table: service_request

request_id	student_id	service_type_id	description	request_date	status	resolved_on
1	1	1	Ceiling fan not working	2026-09-01	Resolved	2026-09-02
2	2	4	Wi-Fi keeps dropping in room A101	2026-09-05	In Progress	
3	3	6	Weekly laundry pickup was missed	2026-09-08	Resolved	2026-09-10
4	4	3	Room cleaning needed	2026-09-12	Assigned	
5	5	5	Request for Jain meal option	2026-09-15	Open	
6	1	2	Leaking tap in the bathroom	2026-09-20	Open	
7	7	1	Tube light flickering near the desk	2026-10-03	Open	

Requests 1 and 3 are Resolved, 2 is In Progress and 4 is Assigned. These statuses and the resolved_on dates were not inserted directly; they were set by trg_update_after_insert from the rows of request_update below.

Table: request_update

update_id	request_id	staff_id	new_status	remarks	updated_on
1	1	6	Assigned	Electrician assigned	2026-09-01 10:30:00
2	1	6	In Progress	Checking fan capacitor	2026-09-01 16:00:00
3	1	6	Resolved	Capacitor replaced, fan works	2026-09-02 11:15:00
4	2	9	Assigned	Network team informed	2026-09-05 12:00:00
5	2	9	In Progress	Access point being replaced	2026-09-06 09:45:00
6	3	8	Assigned	Laundry vendor contacted	2026-09-08 14:00:00
7	3	8	In Progress	Pickup rescheduled	2026-09-09 08:30:00
8	3	8	Resolved	Clothes collected and returned	2026-09-10 18:00:00
9	4	8	Assigned	Housekeeping slot booked	2026-09-12 15:20:00

Table: fee_payment

payment_id	student_id	fee_month	amount	payment_date	payment_mode
1	1	2026-08	4500.00	2026-08-05	UPI
2	2	2026-08	4500.00	2026-08-06	Card
3	3	2026-08	4500.00	2026-08-05	NetBanking
4	4	2026-08	6500.00	2026-08-07	UPI
5	5	2026-08	6500.00	2026-08-10	Cash
6	7	2026-08	4800.00	2026-08-08	UPI
7	8	2026-08	4500.00	2026-08-03	UPI
8	1	2026-09	4500.00	2026-09-04	UPI
9	2	2026-09	4500.00	2026-09-05	UPI

payment_id	student_id	fee_month	amount	payment_date	payment_mode
10	3	2026-09	4500.00	2026-09-06	Card
11	4	2026-09	6500.00	2026-09-06	UPI
12	5	2026-09	6500.00	2026-09-09	Cash
13	7	2026-09	4800.00	2026-09-07	NetBanking
14	1	2026-10	4500.00	2026-10-03	UPI
15	3	2026-10	4500.00	2026-10-04	UPI

8. Workflow of the System

Room allotment.

1. The warden picks a student and a room. sp_allocate_room inserts the allocation.
2. The trigger checks the room's free beds, the hostel type against the student's gender, and whether the student already has a room. Any failure stops the insert with a clear message.
3. When a student leaves, the allocation is marked Vacated with a check-out date. The bed is free at once and the stay stays on record.

Service request.

1. The student raises a request. sp_raise_request checks that the student is a resident and has no open request of the same type, then saves it with status Open.
2. Staff of the matching trade move it to Assigned, then In Progress, then Resolved. Each step is a new row in request_update with remarks and time.
3. The database allows only the next step. A request cannot be resolved before work has started, and a closed request cannot be changed.
4. On Resolved, the trigger fills in resolved_on, which is used to compare the actual time with the target days of the service type.

The student can cancel a request while it is still Open. Because every step is saved in request_update, the history of each complaint is complete.

Rent. A resident pays the rent of their room for a month. The trigger rejects a wrong amount and the unique key rejects a second payment for the same month. The warden's dues report lists every resident with Paid or Due for the chosen month.

9. User Interface

The web interface is connected to the MySQL database through a Node.js server. A user chooses who to log in as: a student, a warden or a member of the service staff, and sees only the screens for that role. Every action is a real SQL statement, and the page shows the statement that was sent to MySQL. When MySQL refuses something, the error message from the trigger or constraint is shown on the page.

Role	Screens
Student	My Room (room, roommates, stay history); Service Requests (raise, track with history, cancel); Fees (pay rent, payment history)
Warden	Allocate Room (allot and vacate); Occupancy with free beds; Fee Dues for any month; All Requests
Service staff	Service Queue for their trade, with a button for the next step and remarks
Everyone	Records (insert and delete rows in any table); Database (row counts, schema, views and queries, activity log)

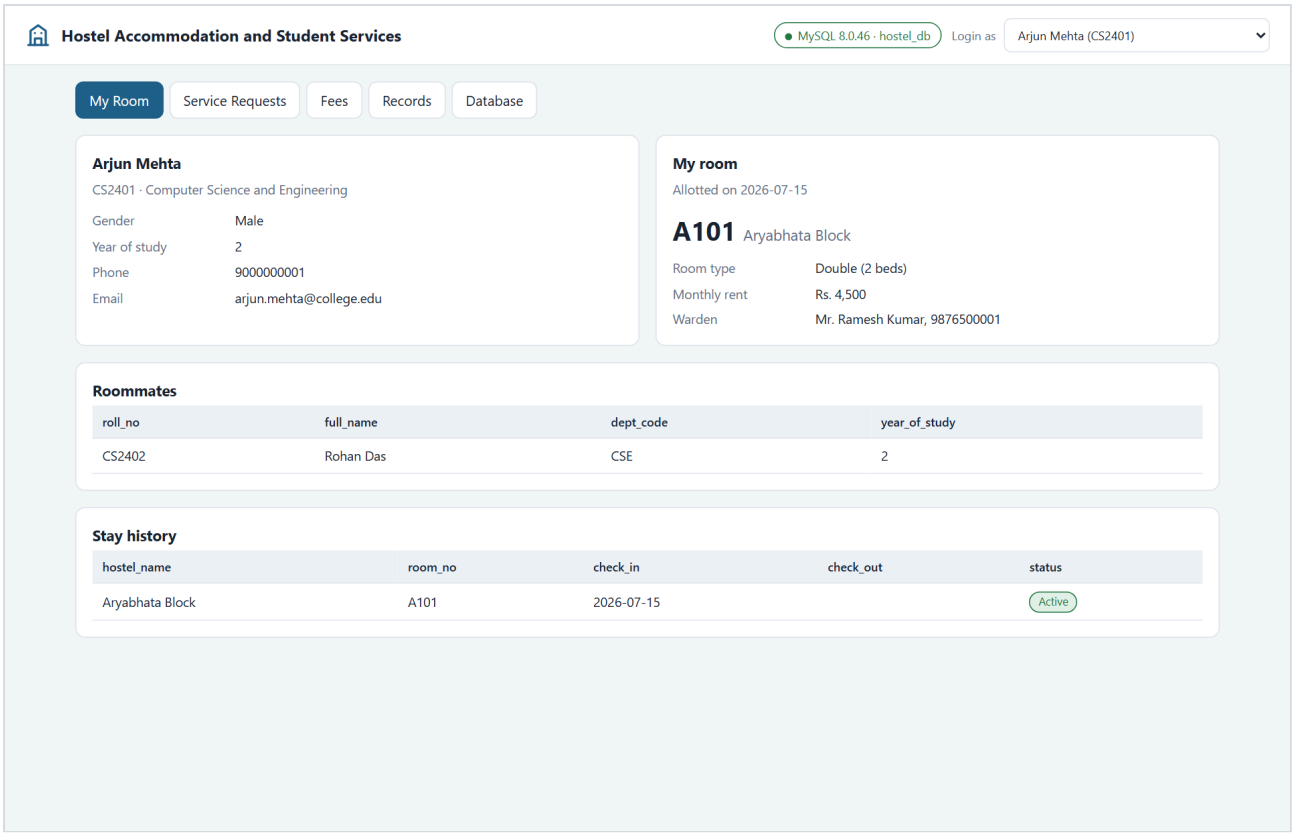


Figure 9.1: Student view of the allotted room

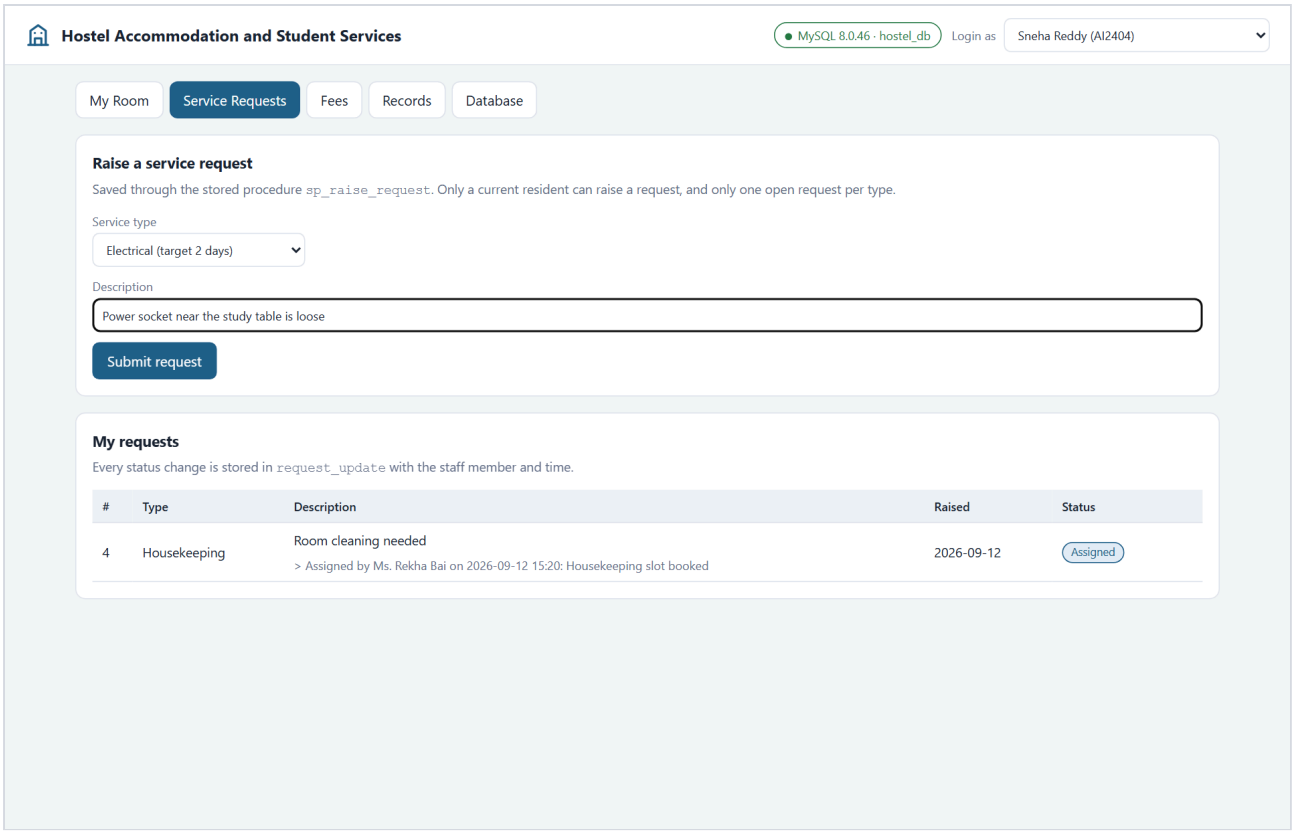


Figure 9.2: Service request form

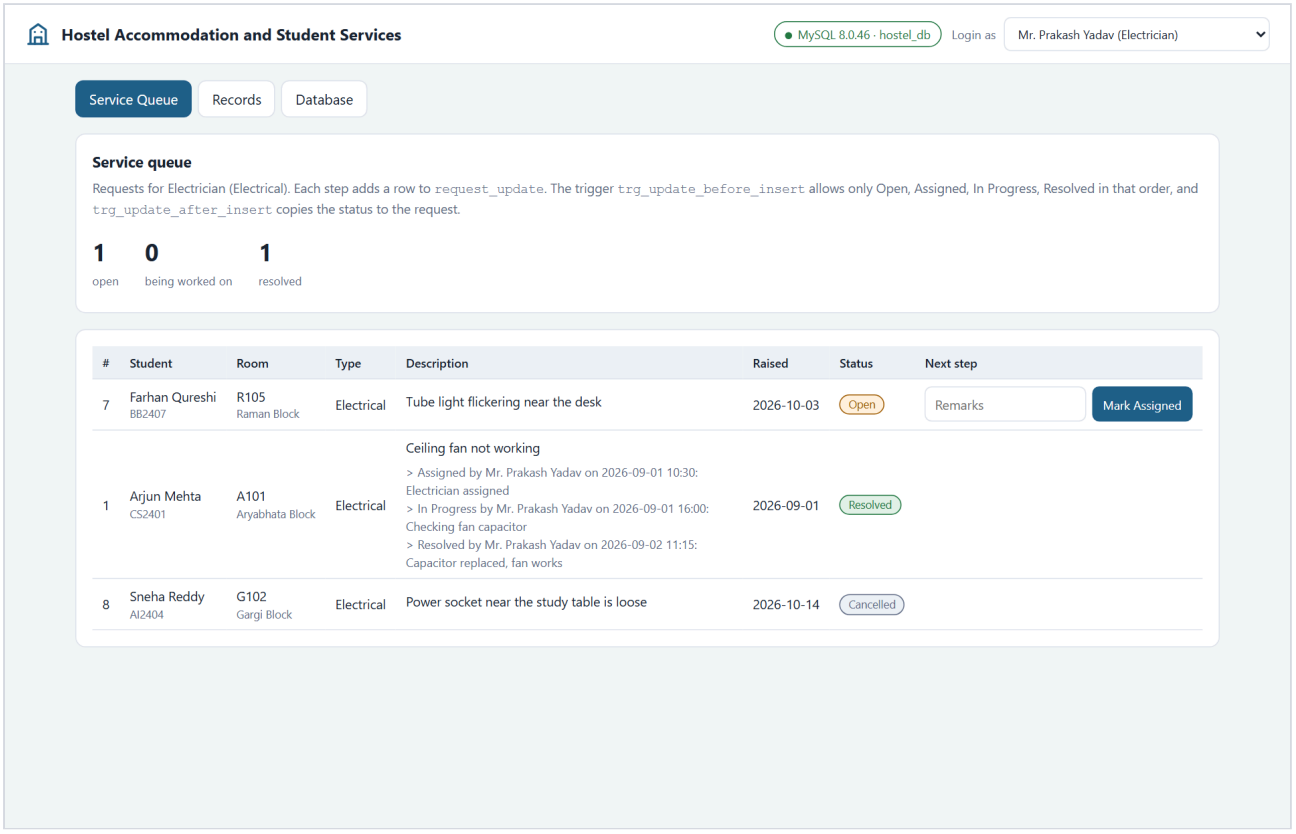


Figure 9.3: Electrician's service queue

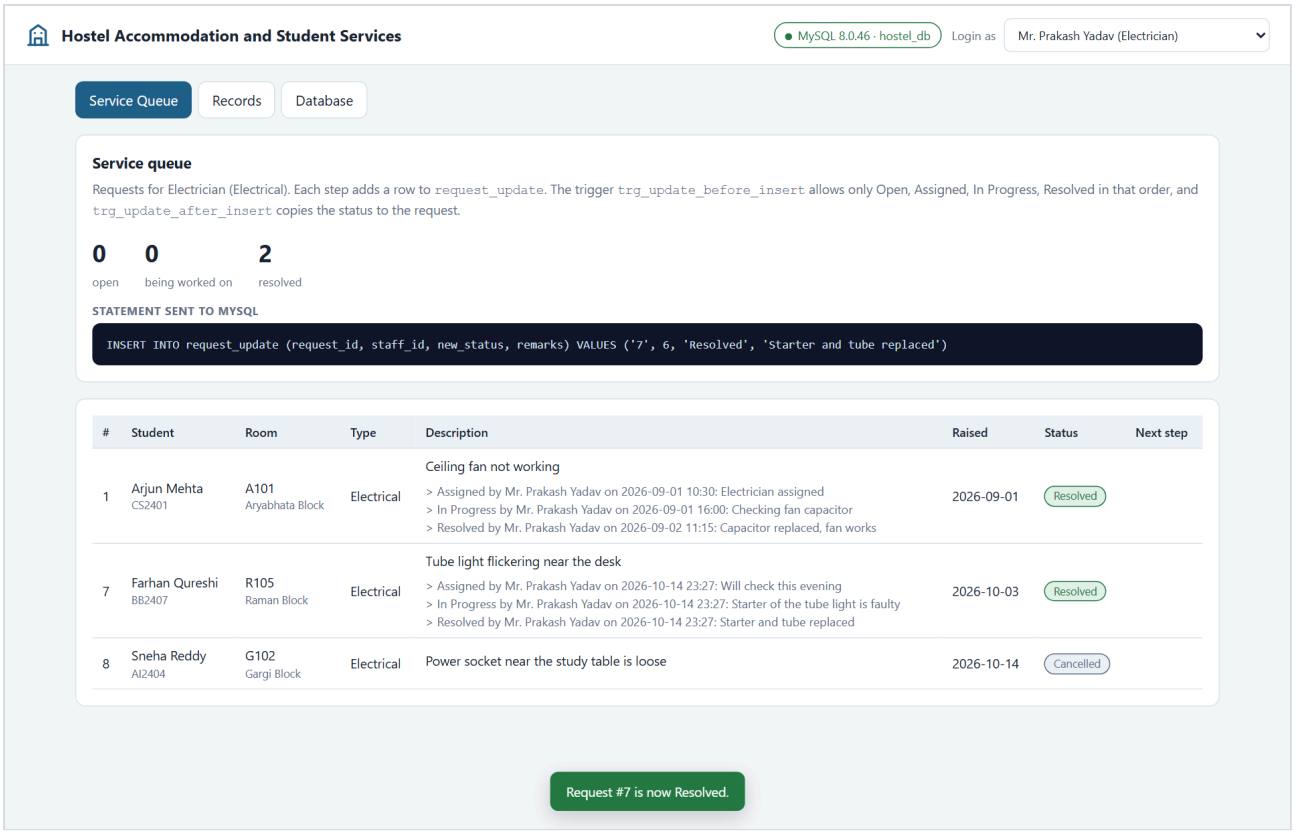


Figure 9.4: Request resolved after the last step

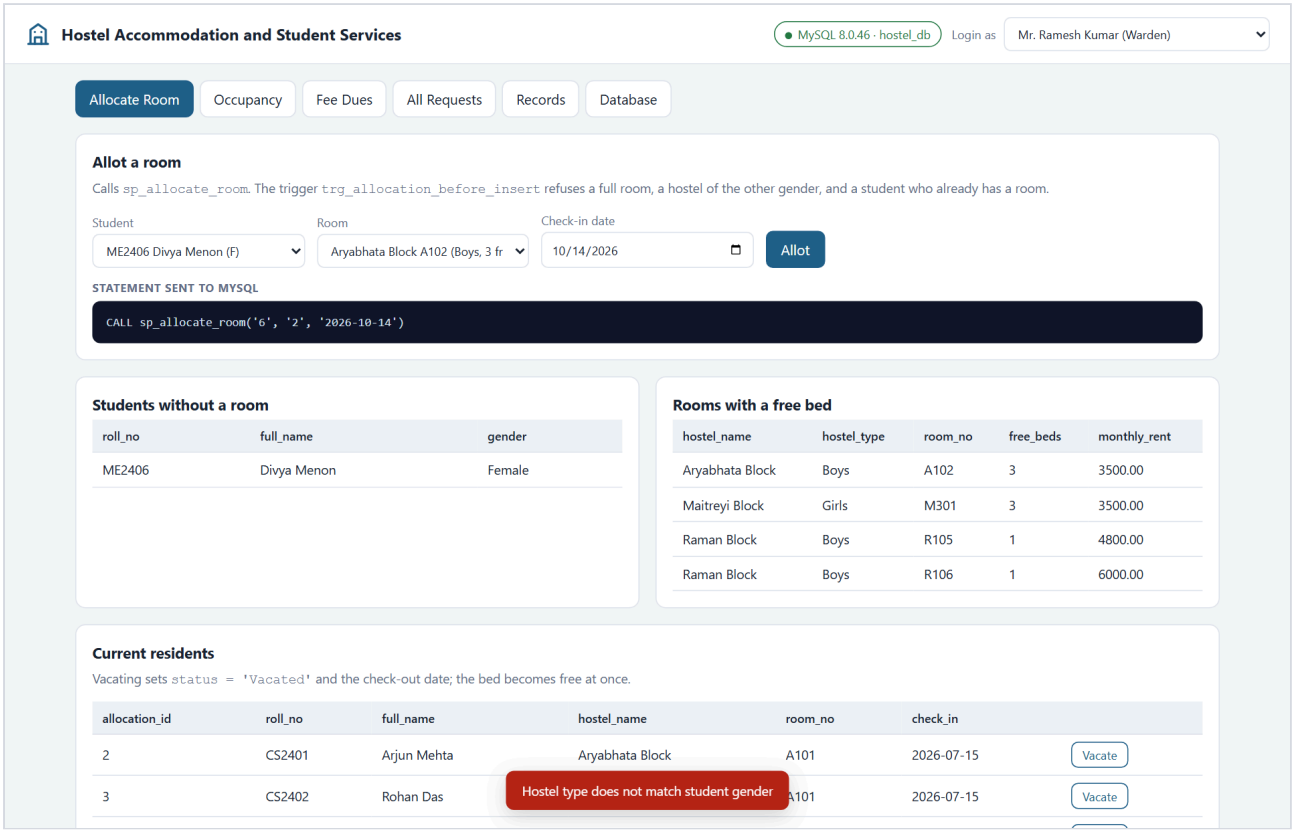


Figure 9.5: Warden refused a room in a hostel of the other gender

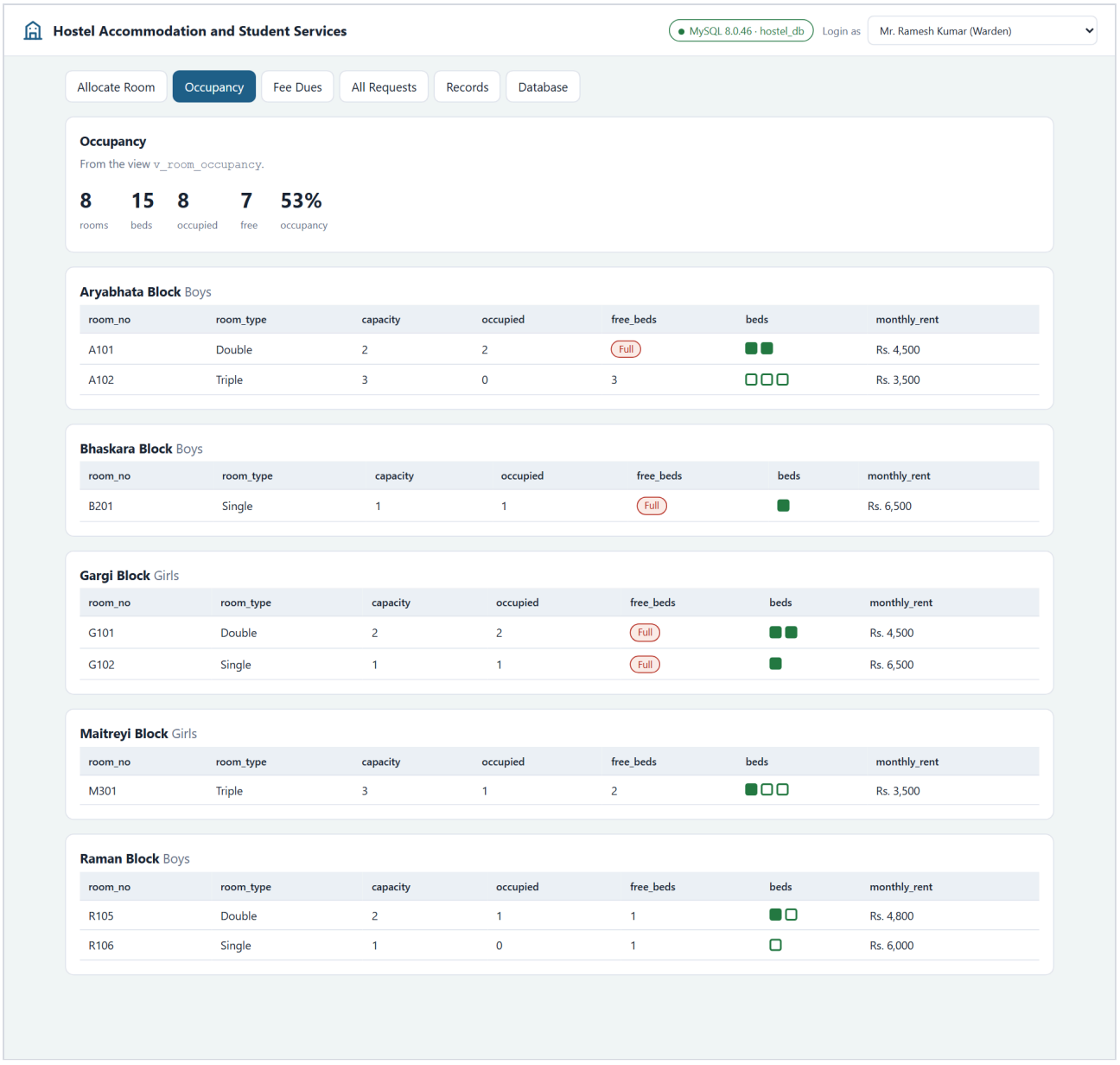


Figure 9.6: Occupancy page built from v_room_occupancy

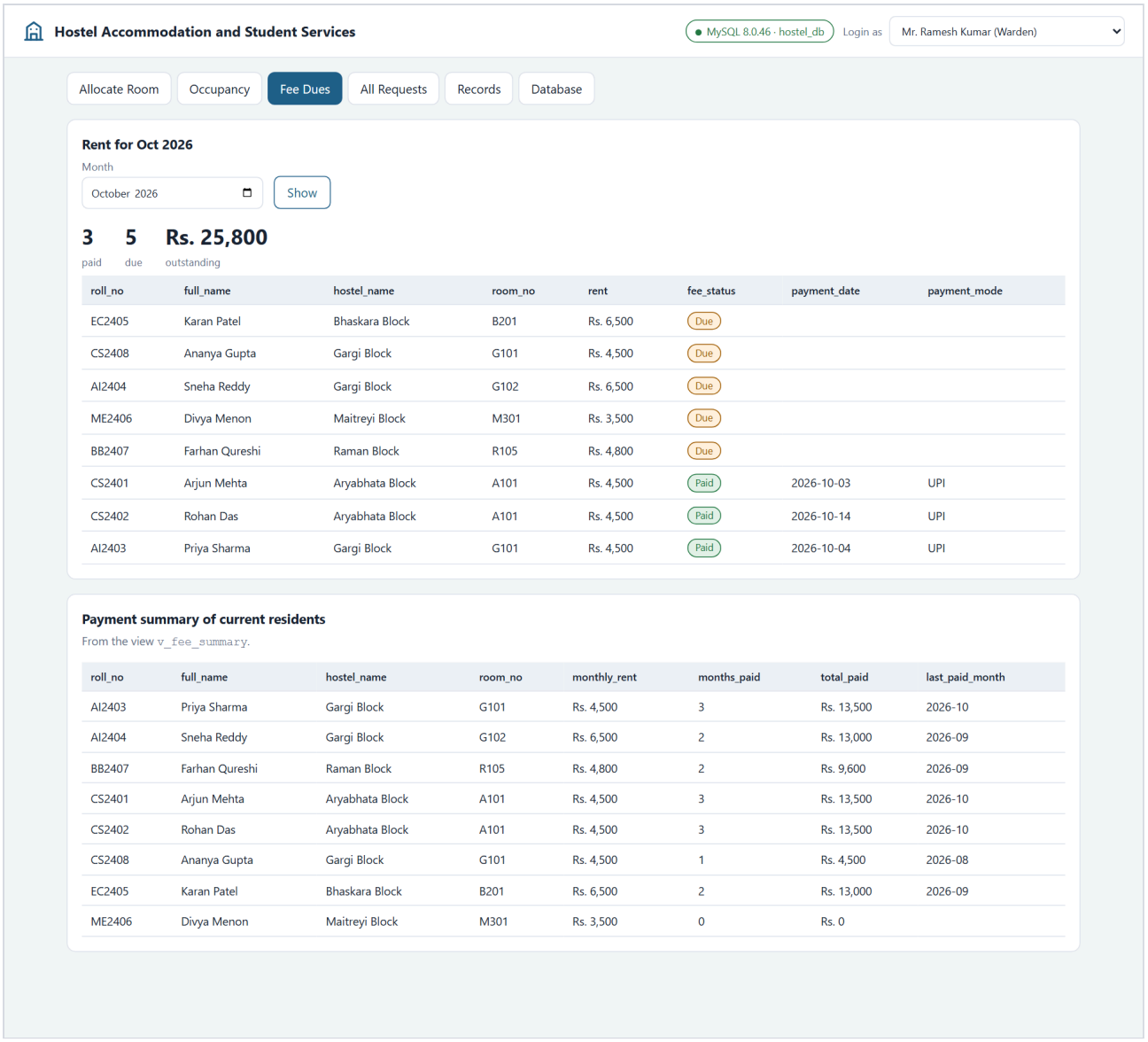


Figure 9.7: Fee dues for October 2026

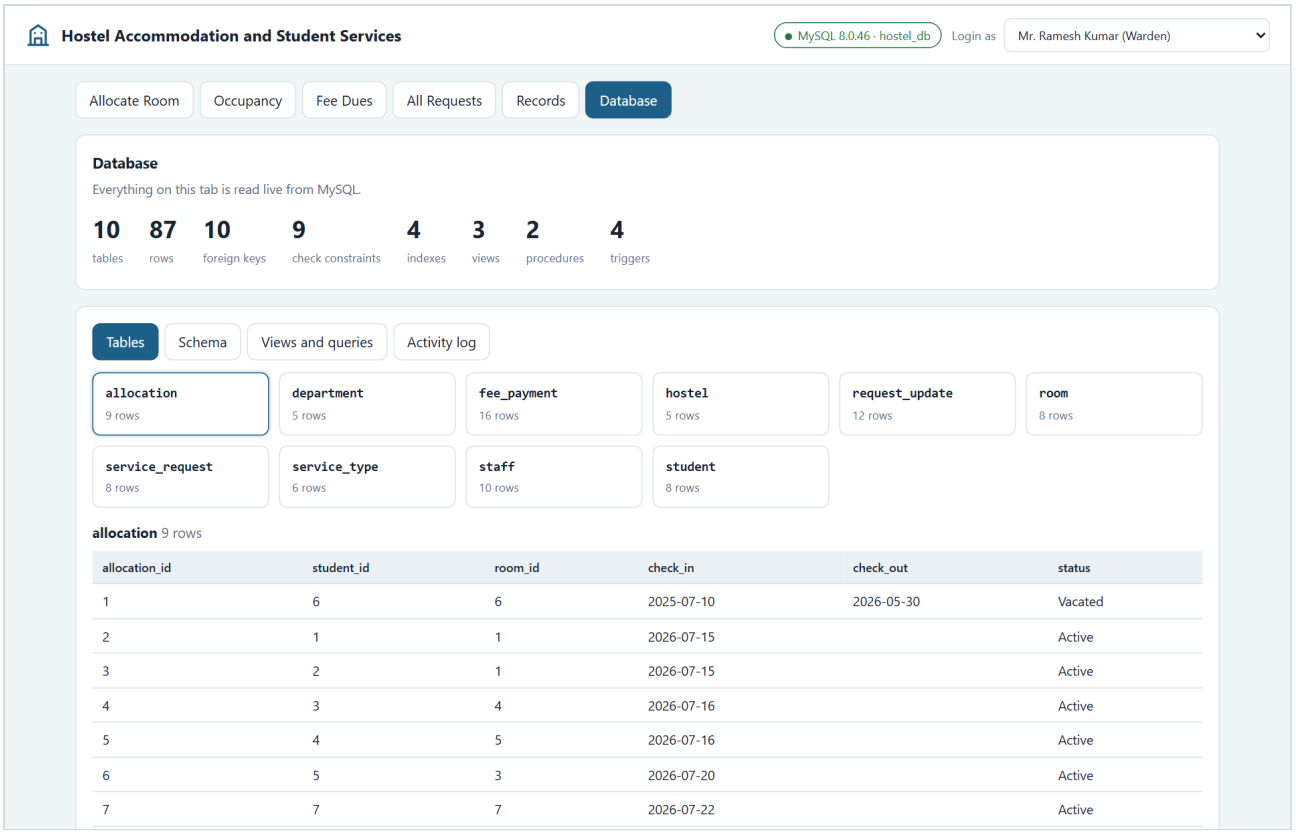


Figure 9.8: Database tab with live row counts

Screenshots of every screen, each insert, delete and view shown before and after with the matching MySQL output, are in Presentation/UI_Screenshots_Hostel_Accommodation_Student_Services.pdf.

10. Testing and Results

The sample data has eight students, ten staff members, five hostels with eight rooms, seven service requests and fifteen rent payments. The tests below check the rules of the system. Each one was run on MySQL 8.0 and gave the expected result.

No.	Test	Expected result	Rule checked
1	Allot a female student to a room in a boys' hostel	Rejected: "Hostel type does not match student gender"	BEFORE trigger
2	Allot a student to G101, which is full	Rejected: "Room is full"	BEFORE trigger
3	Allot a second room to a student who has one	Rejected: "Student already has an active room"	BEFORE trigger
4	Insert a room with capacity 5	Rejected: check constraint chk_room_capacity	CHECK constraint
5	Raise a request for a student with no room	Rejected: "Only a current hostel resident can raise a request"	Procedure

No.	Test	Expected result	Rule checked
6	Raise a second open Plumbing request	Rejected: "An open request of this type already exists"	Procedure
7	Mark an Open request as Resolved	Rejected: "Request must be In Progress before it is resolved"	BEFORE trigger
8	Add a step to a Resolved request	Rejected: "Service request is already closed"	BEFORE trigger
9	Move a request to Assigned, In Progress, Resolved	Status follows each step; resolved_on is filled in	AFTER trigger
10	Pay Rs. 4,000 for a room with rent Rs. 4,500	Rejected: "Amount must equal the monthly rent of the room"	BEFORE trigger
11	Pay the same month twice	Rejected: duplicate entry for (student_id, fee_month)	Unique constraint
12	Delete a hostel that still has rooms	Rejected: foreign key constraint fails	Foreign key
13	Delete a student	Student's allocations, requests and payments are removed	ON DELETE CASCADE
14	Allot Divya Menon to M301	Accepted; M301 shows 1 of 3 beds taken	Procedure and view

10.1 Result with sample data

After the script runs, 7 of the 15 beds are taken, so occupancy is 47%. A101, B201, G101 and G102 are full, and the only girls' room with a free bed is M301. Of the seven service requests, two are Resolved, one is In Progress, one is Assigned and three are Open. For October 2026, only Arjun Mehta and Priya Sharma have paid, so the dues query (Q6) lists the other five residents.

In the interface walkthrough the electrician took Farhan Qureshi's request #7 from Open to Resolved in three steps, and the trigger set its resolved_on date. Divya Menon was refused a room in Aryabhata Block (wrong gender) and in G101 (full), and was then allotted M301. Rohan Das's payment of Rs. 4,000 was refused and his payment of Rs. 4,500 was accepted.

Measure	Before walkthrough	After walkthrough	Reason
Beds taken	7 of 15 (47%)	8 of 15 (53%)	Divya allotted M301
Resolved requests	2	3	Request #7 resolved
October rent paid	2 of 7 residents	3 of 8 residents	Rohan paid; Divya added as a resident

The report queries (Appendix A) were used to list free beds, open requests with their history, residents with rent due and collection per hostel. Q8 shows that the resolved Electrical request took 1 day against a target of 2, and the Laundry request took 2 days against a target of 2.

11. Conclusion and Future Scope

This project built a normalized relational database that brings hostel rooms, allotment, service requests and rent into one system. Because the rules are written as constraints, procedures and triggers, they hold however the data is entered, from the web interface, from MySQL Workbench or from a script. A bed cannot be given twice, every complaint has a complete history, and the list of residents with rent due is one query away.

Possible extensions:

- Secure login with passwords and role-based access instead of choosing a user from a list.
- Email or SMS alerts when a request changes status or rent is overdue.
- Online rent payment through a payment gateway, with automatic late fees.
- Mess menu, mess attendance and mess bill tables.
- Visitor log and room change requests, and dashboards for the hostel office.

References

1. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts. McGraw-Hill.
2. Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
3. Oracle Corporation. MySQL 8.0 Reference Manual: CREATE TRIGGER, CREATE PROCEDURE, SIGNAL, CHECK constraints and views. dev.mysql.com/doc.
4. Express documentation, expressjs.com, and mysql2 documentation, sidorares.github.io/node-mysql2.
5. Course lecture notes, Database Management Systems, Woxsen University.

Appendix A: Report Queries

Query 1: occupancy and free beds in every room.

```
SELECT hostel_name, room_no, room_type, capacity, occupied, free_beds
FROM v_room_occupancy
ORDER BY hostel_name, room_no;
```

Query 2: rooms with a free bed in girls' hostels, cheapest first.

```
SELECT hostel_name, room_no, room_type, free_beds, monthly_rent
FROM v_room_occupancy
WHERE hostel_type = 'Girls' AND free_beds > 0
ORDER BY monthly_rent;
```

Query 3: who stays where.

```
SELECT s.roll_no, s.full_name, d.dept_code, h.hostel_name, r.room_no, a.check_in
FROM allocation a
JOIN student s    ON s.student_id = a.student_id
JOIN department d ON d.dept_id = s.dept_id
JOIN room r       ON r.room_id = a.room_id
JOIN hostel h     ON h.hostel_id = r.hostel_id
WHERE a.status = 'Active'
ORDER BY h.hostel_name, r.room_no;
```

Query 4: open service requests with the student's room.

```
SELECT request_id, full_name, hostel_name, room_no, type_name, request_date, status
FROM v_open_requests
ORDER BY request_date;
```

Query 5: history of every request (who changed the status and when).

```
SELECT sr.request_id, s.full_name, st.type_name, ru.new_status,
       f.full_name AS staff, ru.remarks, ru.updated_on
FROM service_request sr
JOIN student s       ON s.student_id = sr.student_id
JOIN service_type st ON st.service_type_id = sr.service_type_id
JOIN request_update ru ON ru.request_id = sr.request_id
JOIN staff f         ON f.staff_id = ru.staff_id
ORDER BY sr.request_id, ru.updated_on;
```

Query 6: residents who have not paid rent for October 2026.

```
SELECT s.roll_no, s.full_name, h.hostel_name, r.room_no, r.monthly_rent AS due
FROM allocation a
JOIN student s ON s.student_id = a.student_id
JOIN room r    ON r.room_id = a.room_id
JOIN hostel h  ON h.hostel_id = r.hostel_id
WHERE a.status = 'Active'
  AND NOT EXISTS (SELECT 1 FROM fee_payment f
                  WHERE f.student_id = a.student_id AND f.fee_month = '2026-10');
```

Query 7: rent collected per hostel per month.

```
SELECT h.hostel_name, f.fee_month, COUNT(*) AS payments, SUM(f.amount) AS collected
FROM fee_payment f
JOIN allocation a ON a.student_id = f.student_id AND a.status = 'Active'
JOIN room r       ON r.room_id = a.room_id
```

```
JOIN hostel h     ON h.hostel_id = r.hostel_id
GROUP BY h.hostel_name, f.fee_month
ORDER BY f.fee_month, h.hostel_name;
```

Query 8: resolved requests per service type against the target time.

```
SELECT st.type_name, st.target_days, COUNT(*) AS resolved,
       AVG(DATEDIFF(sr.resolved_on, sr.request_date)) AS avg_days
FROM service_request sr
JOIN service_type st ON st.service_type_id = sr.service_type_id
WHERE sr.status = 'Resolved'
GROUP BY st.service_type_id, st.type_name, st.target_days;
```

Query 9: hostel residents per department (departments with at least two).

```
SELECT d.dept_name, COUNT(*) AS residents
FROM allocation a
JOIN student s    ON s.student_id = a.student_id
JOIN department d ON d.dept_id = s.dept_id
WHERE a.status = 'Active'
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(*) >= 2;
```

Appendix B: Source Code

The SQL script, the user interface source code, the presentation and this report are in the project repository:

https://github.com/amaancoderx/DBMS-Course-Project